

112590024 邱紹澹 作業系統作業 HW3

7.8: The Linux kernel has a policy that a process cannot hold a spinlock while attempting to acquire a semaphore. Explain why this policy is in place.

7.8

因為持有 Spinlock 的行程「絕對不能休眠」，而嘗試獲取 Semaphore 失敗時會強迫「進入休眠」。

若帶著 Spinlock 進入休眠，會導致其他等待 Spinlock 的程式在 CPU 上不斷空轉，最後造成系統 Deadlock。

8.20: In a real computer system, neither the resources available nor the demands of processes for resources are consistent over long periods (months). Resources break or are replaced, new processes come and go, and new resources are bought and added to the system.

If deadlock is controlled by the banker's algorithm, which of the following changes can be made safely (without introducing the possibility of deadlock), and under what circumstances?

- (a) Increase Available (new resources added).
- (b) Decrease Available (resource permanently removed from system).
- (c) Increase Max for one process (the process needs or wants more resources than allowed).
- (d) Decrease Max for one process (the process decides that it does not need that many resources).
- (e) Increase the number of processes.
- (f) Decrease the number of processes.

8.20

(a)

絕對安全。資源變多只會讓系統更有餘裕，原本的 safe sequence 還是成立。

(b)

不一定安全。少了資源，原本的 safe sequence 可能跑不下去，必須重新跑 Safety Algorithm 確認還能找到 safe sequence 才行。

(c)

不一定安全。Max 變大代表 Need 也變大，行程未來可能跟系統要更多資源，必須重新跑 Safety Algorithm 確認系統還在 safe state。

(d)

安全。但前提是新的 Max 不能比該行程已經拿到的 Allocation 還少。Need 變小對系統只有好處。

(e)

不一定安全。新行程加入時會帶來新的資源需求，必須重新跑 Safety Algorithm，確認加入後系統還能找到 safe sequence。

(f)

安全。行程離開會把它佔用的資源還給系統，Available 增加，系統只會變得更安全。

Answer the following questions using the banker' s algorithm.

(a) Illustrate that the system is in a safe state by demonstrating an order in which the threads may complete.

(d) If a request from thread T3 arrives for (2, 2, 1, 2), can the request be granted immediately? Show the reasons why.

8.28

	Allocation A B C D	Max A B C D	Available A B C D
T0	3 1 4 1	6 4 7 3	2 2 2 4
T1	2 1 0 2	4 2 3 2	
T2	2 4 1 3	2 5 3 3	
T3	4 1 1 0	6 3 3 2	
T4	2 2 2 1	5 6 7 5	

8_28.png

a) NCU
 T_0 3332
 T_1 2130
 T_2 0120
 T_3 2222
 T_4 3454
 $\boxed{2224} \rightarrow T_2 \rightarrow 4117 \rightarrow$
 $T_1 \rightarrow 6779 \rightarrow T_3 \rightarrow$
 $10, 8, 8, 9 \rightarrow T_0 \rightarrow 13, 12, 12, 10$
 $\rightarrow T_4 \rightarrow 13$
 safe sequence
 $\Rightarrow T_2 \rightarrow T_1 \rightarrow T_3 \rightarrow T_0 \rightarrow T_4$

b)
 T_0 3312
 T_1 2130
 T_2 0120
 T_3 3222 - 2212 - 1010
 T_4 3454
 $T_3 \rightarrow T_2 \rightarrow T_1 \rightarrow T_0 \rightarrow T_4$

8_28ans.jpg

8.30: A single-lane bridge connects the two Vermont villages of North Tunbridge and South Tunbridge. Farmers in the two villages use this bridge to deliver their produce to the neighbor town.

The bridge can become deadlocked if a northbound and a southbound farmer get on the bridge at the same time. (Vermont farmers are stubborn and are unable to back up.)

Using semaphores and/or mutex locks, design an algorithm in pseudocode that prevents deadlock.

Initially, do not be concerned about starvation (the situation in which northbound farmers prevent southbound farmers from using the bridge, or vice versa).

8.30

Pseudocode

```
mutex bridge_lock = 1
```

```
northbound_farmer():
```

```
    wait(bridge_lock)
```

```
    cross_bridge()
```

```
    signal(bridge_lock)
```

```
southbound_farmer():
```

```
    wait(bridge_lock)
```

```
    cross_bridge()
```

```
    signal(bridge_lock)
```

一個一個過，用bridge_lock鎖住，來讓一邊先通行。

9.13: Given six memory partitions of 100 MB, 170 MB, 40 MB, 205 MB, 300 MB, and 185 MB (in order), how would the first-fit, best-fit, and worst-fit algorithms place processes of size 200 MB, 15 MB, 185 MB, 75 MB, 175 MB, and 80 MB (in order)?

Indicate which—if any—requests cannot be satisfied.

Comment on how efficiently each of the algorithms manages memory.

9.13

分區大小：100, 170, 40, 205, 300, 185 MB

First-Fit（找第一個夠大的）

Process	Size	分配到	剩餘
P1	200 MB	205 MB	5 MB
P2	15 MB	100 MB	85 MB
P3	185 MB	300 MB	115 MB
P4	75 MB	170 MB	95 MB
P5	175 MB	185 MB	10 MB
P6	80 MB	95 MB	15 MB

全部滿足。

Best-Fit（找最小但夠大的）

Process	Size	分配到	剩餘
P1	200 MB	205 MB	5 MB

Process	Size	分配到	剩餘
P2	15 MB	40 MB	25 MB
P3	185 MB	300 MB	115 MB
P4	75 MB	85 MB	10 MB
P5	175 MB	185 MB	10 MB
P6	80 MB	115 MB	35 MB

全部滿足。

Worst-Fit (找最大的)

Process	Size	分配到	剩餘
P1	200 MB	300 MB	100 MB
P2	15 MB	205 MB	190 MB
P3	185 MB	190 MB	5 MB
P4	75 MB	185 MB	110 MB
P5	175 MB	不足，無法分配	-
P6	80 MB	110 MB	30 MB

P5 無法滿足。

效率比較

- **First-Fit**：速度最快，從頭找到第一個夠用的就分配，但前面的分區容易產生碎片
- **Best-Fit**：浪費空間最少，但會產生很多小碎片難以再利用
- **Worst-Fit**：保留較大剩餘空間給之後使用，但本題反而造成 P5 無法分配，效率最差

9.24: Consider a computer system with a 32-bit logical address and 8-KB page size. The system supports up to 1 GB of physical memory. How many entries are there in each of the following?

- (a) A conventional, single-level page table
 (b) An inverted page table

9.24

(a)

Page number bits = $32 - 13 = 19$ bits

entries = $2^{19} = 524,288$ entries

(b)

Inverted page table 是以實體頁框為單位，每個頁框對應一筆 entry。

Physical frames = $2^{30} / 2^{13} = 2^{17} = \mathbf{131,072 \text{ entries}}$